

LECTURE 2

Computing Environments and Interrupts

CSC323 · Operating Systems

COMSATS University Islamabad, Lahore Campus

In This Lecture

- The major computing environments in use today, and how each shapes its OS
- Why interrupt-driven operation, not polling, lets an OS react instead of check
- The interrupt vector table, and how the CPU finds the right handler without searching
- Software interrupts (traps) vs. hardware interrupts
- The four main interrupt categories: I/O completion, timer, program error, system call

Traditional (Desktop) and Mobile Computing

- **Traditional** — “a computer” used to mean one standalone PC; that line has blurred as desktops and laptops stay always connected and lean on remote servers
- **Battery power** — every scheduling and I/O decision on mobile has a direct, measurable energy cost
- **Limited memory** — mobile devices carry far less RAM, so the OS aggressively suspends or kills background apps
- **Wireless reliance** — Wi-Fi, cellular, Bluetooth, and NFC replace the wired connections a desktop could assume
- **Different sensors** — GPS, accelerometers, gyroscopes, and touchscreens are first-class inputs a desktop OS never had to manage

Client-Server and Peer-to-Peer Computing

- **Client-server** — centralizes a service on one or more powerful servers that satisfy requests from many clients — a compute server or a file server
- **This is what today's internet runs on** — not the old dumb-terminal/mainframe model; the client is capable but delegates specific services
- **Peer-to-peer (P2P)** — removes the client/server distinction entirely; every node is a peer, acting as both client and server
- **Locating a service** — a peer broadcasts a request, or queries a centralized lookup service, before direct peer-to-peer communication begins; BitTorrent is the canonical example

Cloud Computing

Delivers computing, storage, and application services over a network as an on-demand, metered utility rather than owned hardware — built on virtualization underneath.

Public cloud

Available to anyone willing to pay, over the open internet (AWS, Azure, and similar)

Private cloud

Run by an organization strictly for its own internal use, on hardware it controls

Hybrid cloud

Keeps sensitive workloads private while bursting elastic demand out to a public cloud

A cloud environment is physical hardware running a virtual machine manager, plus provisioning, billing, and monitoring software — each application can get its own VM, created and destroyed on demand.

Five Computing Environments

Traditional

Standalone desktop/laptop, now almost always networked

Mobile

Battery, memory, and wireless constraints shape every decision

Client-Server

Capable clients delegate specific services to a central server

Peer-to-Peer

No fixed server — every node can be both client and server

Cloud

Public, private, or hybrid — built on virtualization underneath

These overlap constantly in practice: a mobile app is almost always a client-server app, talking to a backend in a public or hybrid cloud. Treat these as lenses, not mutually exclusive categories.

Interrupt-Driven Operation: Why Not Just Poll?

- **Polling** — a loop that repeatedly reads a device's status register, asking “are you ready yet?” — every “not yet” wastes a CPU cycle
- **Interrupt-driven operation inverts this** — the device tells the CPU, by asserting a signal the control unit checks after every instruction
- **Until the signal appears** — the CPU is completely free to run other work
- **This is what makes multiprogramming and multitasking possible at all** — handing the CPU to another process depends on the device being able to interrupt later, not on the CPU polling it

The Interrupt Vector Table

A table of addresses, one entry per interrupt type, indexed by a small integer. The instant interrupt N occurs, the CPU reads entry N and jumps directly there — constant-time lookup, no searching.

Vector 0	Timer ISR
Vector 1	Keyboard ISR
Vector 14	Disk ISR
Vector 0x80	System-call gate

Alongside the interrupt number, the hardware preserves enough context to resume afterward — typically the interrupted instruction's address, pushed onto a stack before control transfers to the handler.

Software Interrupts (Traps) vs. Hardware Interrupts

Hardware interrupt

- An electrical signal from a device controller, timer chip, or similar hardware
- Arrives asynchronously — completely independent of whatever instruction the CPU happens to be executing

Software interrupt (trap)

- Generated synchronously by the currently running program, via a dedicated trap instruction
- A deliberate system call (“please do this for me”), or a hardware-detected error: divide-by-zero, invalid memory reference, privileged instruction in user mode

Types of Interrupts

Four categories account for almost every interrupt an OS has to service:

Interrupt type	Raised by	Typical example	Synchronous?
I/O completion	A device controller	A disk read finishes; a network packet arrives	Asynchronous (hardware)
Timer	A hardware clock, ticking at a fixed interval	The scheduler's quantum expires, forcing a context switch	Asynchronous (hardware)
Program error / trap	The CPU itself, detecting a fault	Divide-by-zero; illegal memory reference; privileged instruction in user mode	Synchronous (software)
System call	The running program, deliberately	A program calls <code>read()</code> and needs the kernel to perform it	Synchronous (software)

The Timer Interrupt: Preemptive Multitasking's Foundation

Without it, one process could run forever

The timer interrupt is what makes preemptive multitasking possible at all. Without a periodic, unavoidable timer interrupt forcing the OS back into control, a process that never performs I/O and never voluntarily yields the CPU could simply run forever, starving every other process on the system.

Connecting this back to Lecture 1

Lecture 1's dual-mode execution model and this lecture's interrupt types are two halves of the same story: an interrupt — of any of the four kinds — is precisely the event that flips the hardware's mode bit back to kernel mode, which is how the OS regains control from a running user program.

Key Takeaways

- Five computing environments — traditional, mobile, client-server, peer-to-peer, and cloud (public, private, or hybrid) — cover most of where software runs today, and real systems routinely combine several at once
- Interrupt-driven operation replaces wasteful polling: devices signal the CPU when ready, freeing it to do other work meanwhile
- The interrupt vector table gives the CPU constant-time dispatch to the correct handler, indexed by interrupt number
- Hardware interrupts arrive asynchronously from devices; software interrupts (traps) are synchronous — a system call or a hardware-detected program error
- I/O completion, timer, program error, and system call together explain almost everything that forces the CPU back into the OS's hands